

Predictive Modeling: Regression

CIS 635 — Knowledge Discovery & Data Mining
Concepts, Model Development, and Evaluation

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [7] this material when using or reproducing it.

Unit: Predictive Modeling — Regression

This reading opens the predictive-modeling unit. It builds directly on the three-part EDA reading — **Basic EDA Concepts** (Part 1), **Advanced EDA Concepts** (Part 2), and **Feature Selection and Dimensionality Reduction** (Part 3) — and on the feature-scaling material. In particular, it reuses correlation (Part 1, Section 1.6), data leakage (Part 1, Section 2.7), the curse of dimensionality (Part 2, Section 3.4), the variance inflation factor and the Lasso as an embedded feature-selection method (Part 3, Sections 1.1 and 1.4), and standardization.

Contents

1	Regression in the KDD Process	4
1.1	Predictive Modeling within KDD	4
1.2	Regression Is a Workflow, Not an Algorithm	5
2	Foundations of Regression	5
2.1	The Regression Problem	5
2.2	Simple Linear Regression	6
2.3	The Least-Squares Principle	6
3	Linear Regression	7
3.1	Simple Linear Regression in Practice	7
3.2	Multiple Linear Regression	8
4	Regression Assumptions and Diagnostics	9
4.1	The Assumptions and Why They Matter	9
4.2	Practical Diagnostic Tools	10
5	Model Complexity and Regularization	11
5.1	Underfitting, Overfitting, and the Bias–Variance Trade-off	11
5.2	Ridge, Lasso, and Elastic Net	12
6	Polynomial and Nonlinear Relationships	13
7	Developing a Regression Model	14

8	Evaluation of Regression Models	15
8.1	Error Metrics: MAE, MSE, and RMSE	15
8.2	Coefficient of Determination: R^2 and Adjusted R^2	16
8.3	Choosing and Reporting Metrics	17
9	Python Implementation	17
10	Integrated KDD Case Study: Predicting Diabetes Progression	21
11	Common Pitfalls	22
12	Key Takeaways	23

Notation and Terminology Quick Reference

The table below collects the recurring terms and symbols of this reading for quick lookup. Each is developed fully in the section listed. Terms from the EDA readings — correlation, data leakage, standardization, VIF, filter/wrapper/embedded selection — appear in those readings’ own quick-reference tables.

Term / symbol	Short definition	§
$y, x_1, \dots, x_p$	Target (response) variable; p predictor (feature) variables	2.1
$\hat{y}_i$	Fitted or predicted value for observation i	2.1
$e_i = y_i - \hat{y}_i$	Residual: what the model failed to explain for observation i	2.1
$b_0, b_1, \dots, b_p$	Estimated intercept and coefficients (model parameters)	2.2
SSE, SST	Sum of squared errors (residuals); total sum of squares around $\bar{y}$	2.3
Ordinary least squares (OLS)	Choosing coefficients that minimize SSE	2.3
Extrapolation	Predicting outside the range of the training data	3.1
Homoscedasticity	Residual spread is roughly constant across fitted values	4.1
Leverage, Cook’s distance	How unusual an observation’s predictors are; how much it moves the fit	4.2
Bias-variance trade-off	Simple models miss structure; flexible models chase noise	5.1
Ridge, Lasso, Elastic Net	Regression with L_2 , L_1 , or mixed coefficient penalties	5.2
λ (alpha)	Penalty strength; a hyperparameter tuned by cross-validation	5.2
MAE, MSE, RMSE	Average absolute / squared / root-squared prediction error	8.1
R^2 , adjusted R^2	Share of target variance explained; version penalized for p	8.2

Learning Objectives

This reading assumes the statistical, preprocessing, EDA, and feature-selection foundations of the earlier readings. After completing it, you should be able to:

- Explain where regression sits in the KDD process and distinguish regression from classification, and prediction from explanation.
- Fit a simple or multiple linear regression model and interpret its intercept, coefficients, fitted values, and residuals, including the “holding other predictors constant” meaning of a coefficient.
- State the main assumptions of linear regression, read residual, Q–Q, leverage, and VIF diagnostics, and explain what each problematic pattern implies for prediction versus interpretation.
- Explain the bias–variance trade-off and choose among ordinary least squares, Ridge, Lasso, Elastic Net, and polynomial features for a given problem, including why features must be scaled before penalization.
- Build a leakage-free regression workflow with a baseline, a training/validation/test protocol, cross-validated hyperparameter tuning, and a single final test evaluation.
- Compute and interpret MAE, MSE, RMSE, R^2 , and adjusted R^2 , explain their units and outlier sensitivity, and explain why a higher R^2 does not by itself mean a better predictive model.
- Communicate regression results responsibly, distinguishing association from causation and acknowledging uncertainty and the limits of the training data.

1 Regression in the KDD Process

1.1 Predictive Modeling within KDD

Fayyad, Piatetsky-Shapiro, and Smyth described knowledge discovery in databases as the overall process of finding valid, novel, useful, and understandable patterns in data, with *data mining* as the step that applies a specific algorithm to extract them [4]. Data-mining tasks are usually split into two broad families [5]. **Descriptive** tasks summarize the structure of the data at hand — the clustering, association, and EDA work of earlier readings. **Predictive** tasks learn a mapping from some variables to another variable so that the mapping can be applied to *new* records whose outcome is not yet known.

Predictive tasks are further divided by the type of the outcome. When the target is categorical (diabetic / not diabetic, fraud / not fraud), the task is **classification**. When the target is a continuous quantity — a price, a dose, a yield, a concentration, a level of soil moisture — the task is **regression**. Table 1 contrasts the two.

Regression is everywhere in applied work. Health systems predict length of stay and treatment cost; precision-agriculture systems predict crop yield or the next day’s soil moisture from weather and sensor data; utilities forecast energy demand; real-estate platforms estimate house prices. The same mathematics also serves a different purpose: *explaining* how an outcome relates to its predictors. Shmueli argues that these two goals — predicting well and explaining well — are genuinely different and can lead to different modeling choices [19]. This reading keeps the distinction in view throughout. A model can predict well while having coefficients that are unstable and hard to interpret, and a model can have perfectly sensible coefficients while predicting poorly.

	Regression	Classification
Target type	Continuous numeric (price, yield, blood glucose)	Categorical label (yes/no, subtype A/B/C)
Typical output	A number, ideally with an uncertainty range	A class, often with a class probability
Error is measured as	Distance between y and $\hat{y}$ (MAE, RMSE)	Agreement of labels (accuracy, precision, recall, AUC)
Example question	“How many days will this patient stay?”	“Will this patient be readmitted?”
Example methods	Linear, Ridge, Lasso, regression trees	Logistic regression, kNN, decision trees

Table 1: Regression versus classification. Many algorithms (trees, kNN, neural networks) have both a regression and a classification version; the target type decides which one applies.

1.2 Regression Is a Workflow, Not an Algorithm

It is tempting to equate “doing regression” with calling `LinearRegression().fit()`. In practice the fitting step is the smallest part of the job. A regression project follows the same KDD arc you have already practiced:

Problem definition → **Data preparation** → **EDA** → **Feature engineering** → **Model development**
→ **Evaluation** → **Interpretation** → **Prediction/Decision**

Every earlier reading feeds into this arc. Correlation analysis (Part 1) suggests which predictors are promising and which are redundant. Handling missing values and leakage (Part 1, Section 2.7) decides whether the evaluation can be trusted. Feature selection (Part 3) and scaling decide what the model sees. Section 7 of this reading returns to the workflow and turns it into concrete steps.

2 Foundations of Regression

2.1 The Regression Problem

A regression dataset has n observations. For each observation i we record a **target** or **response** y_i and a vector of **predictors** or **features** $x_{i1}, \dots, x_{ip}$. The working assumption behind every regression model is

$$Y = f(X) + \epsilon.$$

Here f is the unknown systematic relationship between the predictors X and the target Y , and ϵ is everything f cannot capture: measurement error, unrecorded influences, and genuine randomness [12]. We never observe f ; we *estimate* it from data, producing a fitted model $\hat{f}$. Applying $\hat{f}$ to an observation gives its **fitted value** (on training data) or **predicted value** (on new data), $\hat{y}_i = \hat{f}(x_i)$. The gap between what happened and what the model said is the **residual**,

$$e_i = y_i - \hat{y}_i.$$

A positive residual means the model under-predicted; a negative residual means it over-predicted. Residuals are the raw material for both evaluation (Section 8) and diagnostics (Section 4).

Because ϵ contains randomness, even the true f would not predict perfectly. This *irreducible error* sets a floor on how good any model can be. A realistic goal is therefore not zero error but error close to that floor on data the model has never seen.

2.2 Simple Linear Regression

The simplest choice of f is a straight line in one predictor:

$$\hat{y} = b_0 + b_1x.$$

The **intercept** b_0 is the predicted value when $x = 0$. The **slope** b_1 is the predicted change in y for a one-unit increase in x . b_0 and b_1 are the model's **parameters**, and “fitting” the model means choosing their values.

2.3 The Least-Squares Principle

Infinitely many lines could be drawn through a cloud of points. **Ordinary least squares (OLS)** picks the line with the smallest **sum of squared errors**,

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n e_i^2.$$

Why square the residuals rather than simply add them? Positive and negative residuals would cancel in a plain sum, so a terrible line could have a total of zero. Squaring makes every error count. It also penalizes large misses more than small ones: one residual of 4 contributes 16, while four residuals of 1 contribute only 4. The squared criterion also has a convenient closed-form solution. For simple regression it is

$$b_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad b_0 = \bar{y} - b_1\bar{x}.$$

The slope is the covariance of x and y divided by the variance of x , so it carries the same sign as the Pearson correlation from Part 1, Section 1.6. The intercept formula guarantees that the line passes through the point $(\bar{x}, \bar{y})$.

A small worked example (illustrative values). Five small test plots receive different amounts of fertilizer x (in 100 kg/ha) and produce yield y (t/ha):

Plot	1	2	3	4	5
Fertilizer x	1	2	3	4	5
Yield y	2	4	5	4	6
Fitted $\hat{y} = 1.8 + 0.8x$	2.6	3.4	4.2	5.0	5.8
Residual $e = y - \hat{y}$	-0.6	+0.6	+0.8	-1.0	+0.2

Here $\bar{x} = 3$ and $\bar{y} = 4.2$. The deviations $x_i - \bar{x}$ are $(-2, -1, 0, 1, 2)$ and the deviations $y_i - \bar{y}$ are $(-2.2, -0.2, 0.8, -0.2, 1.8)$. Their products sum to 8.0, and $\sum (x_i - \bar{x})^2 = 10$, so $b_1 = 0.8$ and $b_0 = 4.2 - 0.8 \times 3 = 1.8$. Each additional 100 kg/ha of fertilizer is associated with 0.8 t/ha more yield in these five plots. The residuals sum to zero, as they always do for OLS with an intercept, and $\text{SSE} = 0.36 + 0.36 + 0.64 + 1.00 + 0.04 = 2.4$. No other straight line gives a smaller SSE on these five points. Figure 1 shows the line and the residuals as vertical gaps.

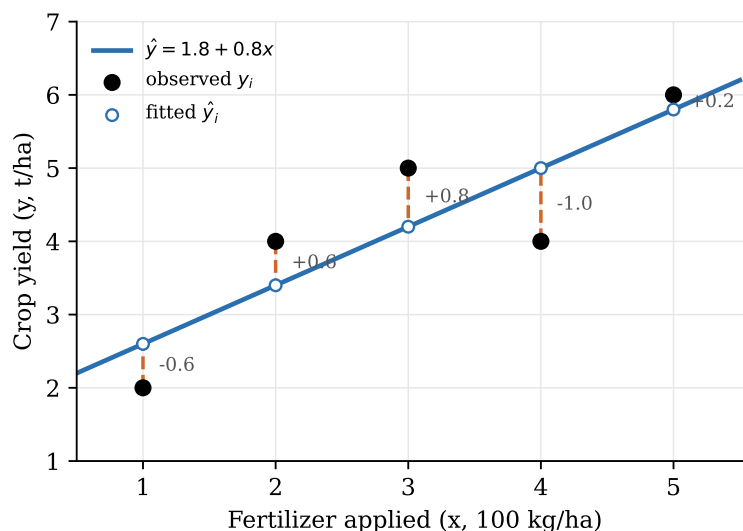


Figure 1: Least-squares fit for the five-plot example (illustrative values). Black dots are observations, open circles are fitted values, and dashed segments are the residuals that OLS makes as small as possible in the squared sense.

Tip

See least squares move. Re-run the five-point example in Python with `np.polyfit(x, y, 1)`, then change the last yield from 6 to 12. The slope jumps from 0.8 to 2.0 because one large residual, once squared, dominates the SSE. This sensitivity to unusual points is the main reason Section 4.2 looks at influential observations.

3 Linear Regression

3.1 Simple Linear Regression in Practice

Once fitted, a line is used for **prediction**: plug in a new x and read off $\hat{y}$. In the fertilizer example, a plot receiving 350 kg/ha ($x = 3.5$) is predicted to yield $1.8 + 0.8 \times 3.5 = 4.6$ t/ha. That is **interpolation**: $x = 3.5$ lies inside the observed range $[1, 5]$. Predicting for $x = 20$ (2,000 kg/ha) gives 17.8 t/ha. That is **extrapolation**, and it is almost certainly wrong, because real crops saturate and fertilizer becomes toxic at high doses. The data say nothing about what happens outside the region they cover. A linear model will nonetheless keep extending the straight line, with complete numerical confidence.

The intercept deserves the same caution. $b_0 = 1.8$ is the predicted yield with no fertilizer. That interpretation is reasonable here, but in many problems $x = 0$ is meaningless (a house with zero square feet) or far outside the data. The intercept is then only an anchor for the line and should not be interpreted.

The deeper limitation of a single predictor is that real outcomes have many drivers. Yield depends on rainfall, soil type, and planting date as well as fertilizer. If those factors happen to vary together with fertilizer in the data, the simple slope silently absorbs their effects. That is the motivation for multiple regression.

3.2 Multiple Linear Regression

With p predictors the model becomes

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \cdots + b_px_p,$$

and OLS again chooses the coefficients that minimize SSE. Geometrically, the fitted model is a flat plane (or hyperplane) instead of a line.

The single most important interpretation rule is this. b_j is the predicted change in y for a one-unit increase in x_j **while all other predictors in the model are held constant** [12]. This is a *partial* effect, and it can differ sharply from the simple correlation of x_j with y . Suppose house prices are modeled from size and number of bedrooms. Holding size fixed, an extra bedroom means the same floor area is cut into smaller rooms, so its partial coefficient may be small or even negative, even though bedrooms and price are positively correlated overall. The coefficient answers a different question from the correlation. Figure 2 summarizes the structure.

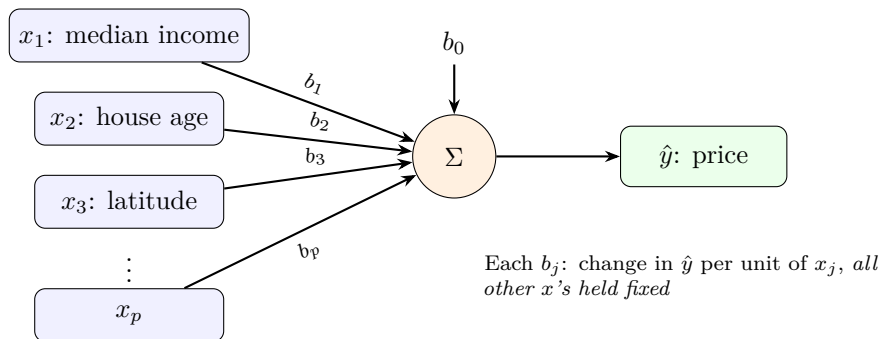


Figure 2: Multiple linear regression as a weighted sum. Each predictor contributes its value times its coefficient; the intercept shifts the total.

Comparing coefficient sizes. Raw coefficients are in the units of their predictors, so they cannot be compared directly: a coefficient per dollar of income and a coefficient per year of age are on different scales. If every predictor is standardized first (mean 0, standard deviation 1, as in the feature-scaling material), each coefficient becomes the predicted change in y for a one-standard-deviation change in that predictor. These standardized coefficients can be compared, at least roughly. The Python example in Section 9 reports coefficients this way.

Categorical predictors. Regression needs numbers, so a categorical predictor such as `region` $\in \{\text{North, Central, South}\}$ is converted to **dummy** (one-hot) indicator columns. With an intercept in the model, one category is left out as the *reference level*. Including all three indicators would make them sum to 1 in every row, duplicating the intercept and making the coefficients non-unique (the “dummy-variable trap”). With North as the reference, the model is

$$\hat{y} = b_0 + b_1 \text{income} + b_2 \mathbb{I}[\text{Central}] + b_3 \mathbb{I}[\text{South}],$$

and b_3 is the predicted difference between South and North for the same income. In scikit-learn, `OneHotEncoder(drop="first")` inside a pipeline handles this, and it learns the category list from the training data only.

Interactions. The basic model assumes each predictor’s effect is the same whatever the values of the others. An **interaction** term such as $b_4(x_1 \times x_2)$ relaxes that assumption: the effect of x_1 becomes $b_1 + b_4x_2$, so it depends on x_2 . Fertilizer might help more in wet years than in dry ones. Interactions are worth adding when domain knowledge or residual patterns suggest them. Adding every pairwise interaction blindly multiplies the number of features and invites overfitting (Section 5).

Common Pitfall

Association is not causation. A regression coefficient describes how the *prediction* changes when a predictor changes with the others held fixed *in the data*. It does not say what would happen if you intervened on that predictor in the world. In the housing data of Section 9, latitude has a large coefficient; moving a house north will not change its value. Latitude simply stands in for coastal location, climate, and local economies. Confounders that are not in the model, reverse causation, and selection effects can all produce large, “significant” coefficients with no causal meaning. Causal claims need a study design built for them (randomization, or explicit causal-inference methods), not a better-fitting regression.

4 Regression Assumptions and Diagnostics

4.1 The Assumptions and Why They Matter

OLS will compute coefficients for any dataset. Whether those numbers mean what we want them to mean depends on a set of assumptions about the data [12]:

1. **Linearity.** The expected value of y is a linear function of the predictors as entered. Transformed or polynomial terms are allowed, since only the coefficients must enter linearly.
2. **Independence.** The errors of different observations are unrelated. Repeated measurements of the same patient, neighboring sensors, or consecutive days usually violate this.
3. **Homoscedasticity.** The spread of the errors is roughly constant across the range of fitted values. Its opposite, *heteroscedasticity*, is common when larger outcomes vary more, as with prices and costs.
4. **Well-behaved residuals.** Residuals are centered on zero without systematic patterns. For classical confidence intervals and p -values they should also be roughly normal.
5. **No severe multicollinearity.** No predictor is (nearly) a linear combination of the others.

How much a violation matters depends on what the model is *for*:

- For **prediction**, the essential conditions are that the linear form is adequate and that new data resemble the training data. Non-normal residuals and moderate collinearity barely affect predictive accuracy. Nonlinearity that the model misses does: it leaves error on the table.
- For **coefficient interpretation**, multicollinearity and omitted variables are the main threats, because they make individual coefficients unstable or misleading even when predictions are fine.
- For **statistical inference** (standard errors, confidence intervals, p -values), independence, homoscedasticity, and approximate normality all matter, since the formulas assume them. This

reading does not develop inference. It is enough to know that violated assumptions make reported p -values unreliable.

4.2 Practical Diagnostic Tools

Figure 3 shows three standard diagnostic plots for an OLS model fitted to the California Housing training data used in Section 9 (16,512 block groups, eight predictors). The patterns are real, and every one of them teaches something.

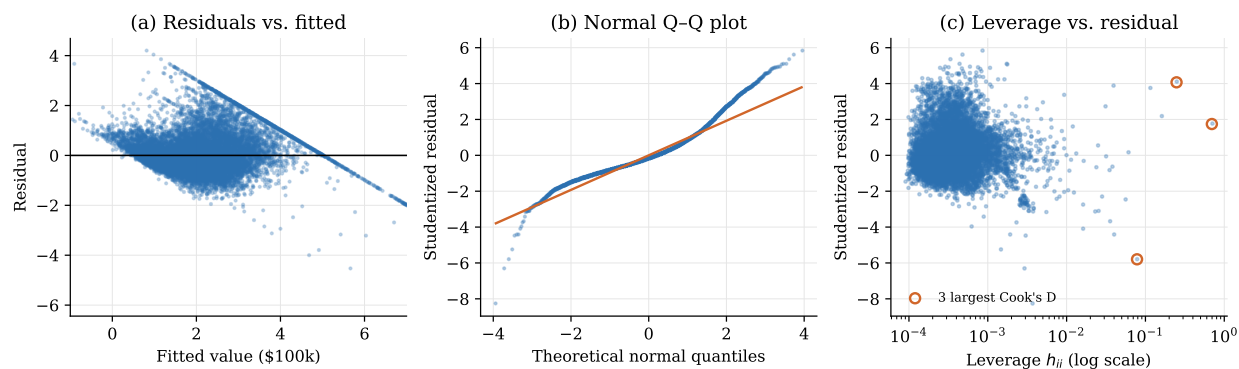


Figure 3: Diagnostics for OLS on the California Housing training set (actual results). (a) Residuals versus fitted values: a fan shape and a sharp diagonal stripe. (b) Q–Q plot of studentized residuals: heavy right tail. (c) Leverage versus studentized residual: a few extreme block groups (circled) have very high leverage.

Residuals versus fitted values. *Look for:* a structureless horizontal band around zero. *Problem patterns:* a curve (U-shape) means a missed nonlinearity; a fan that widens to the right means heteroscedasticity. In panel (a), the spread grows with the fitted value, and a sharp diagonal stripe of points runs downward from upper left to lower right. The stripe is the dataset’s **cap**: about 4.8% of block groups have their median value recorded as exactly \$500,001 [16]. For those points the true value is “at least \$500k,” so the residual $5.0 - \hat{y}$ falls on a straight line of slope -1 . The model cannot learn what the data never recorded. *Remedies:* transformations (e.g., $\log y$ for right-skewed targets), nonlinear terms, or a model suited to censored data. At minimum, report that predictions near the cap are unreliable.

Residual distribution and Q–Q plots. A quantile–quantile (Q–Q) plot places the sorted standardized residuals against the values a normal distribution would produce. Normal residuals fall on the reference line. *Problem patterns:* points curving away at the ends indicate heavy tails or skew. Panel (b) bends upward at the right end, and the residuals have skewness of about 1.1, so the model has occasional large under-predictions. This mainly affects inference and prediction intervals, not the fitted coefficients themselves. *Remedies:* transforming the target, or using robust or quantile-based error estimates.

Leverage and influential observations. An observation’s **leverage** h_{ii} measures how far its predictor values lie from the bulk of the data. **Cook’s distance** combines leverage and residual size to estimate how much the whole fitted model would change if that observation were deleted [2]. A common screening rule flags leverage above $2(p + 1)/n$ or Cook’s distance above $4/n$. With n in the

tens of thousands, these rules flag hundreds of points, so treat them as pointers for inspection, not as deletion criteria. Panel (c) is more telling. The three circled block groups include one with an average of 1,243 occupants per household and one with 142 rooms per household — almost certainly group quarters such as a dormitory or a resort area, or recording artifacts. *Remedies*: investigate before acting. Correct genuine errors; keep genuine but unusual cases and consider transformations that pull them in (Section 9 shows that a log transform does exactly this); never delete points just because they spoil the fit.

Multicollinearity and VIF. Part 3, Section 1.1 introduced the **variance inflation factor**, $VIF_j = 1/(1 - R_j^2)$, where R_j^2 comes from regressing predictor j on all the other predictors. For this model the largest VIFs are latitude (9.2), longitude (8.9), average rooms (7.9), and average bedrooms (6.6): the first pair encodes geography jointly, and the second pair measures dwelling size twice. *Problem*: collinear coefficients have inflated variance, so they swing wildly between samples and can take counter-intuitive signs. Predictions remain fine as long as the predictors stay correlated in the same way in new data. *Remedies*: drop or combine redundant features (e.g., bedrooms *per room*), or use Ridge regression (Section 5). The case study in Section 10 shows a dataset with VIFs near 60.

```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

Xc = sm.add_constant(X_train)          # VIF expects an intercept column
vif = {c: variance_inflation_factor(Xc.values, i)
       for i, c in enumerate(Xc.columns) if c != "const"}

ols = sm.OLS(y_train, Xc).fit()
cooks_d = ols.get_influence().cooks_distance[0] # one value per observation
```

The snippet uses `statsmodels` [18], which complements `scikit-learn` with classical diagnostics.

5 Model Complexity and Regularization

5.1 Underfitting, Overfitting, and the Bias–Variance Trade-off

A model that is too simple misses real structure and has high error on *both* training and new data. This is **underfitting**, or high *bias*. A model that is too flexible fits the training data closely, including its noise, and its predictions change a lot if the training sample changes. This is **overfitting**, or high *variance*. The expected prediction error on new data can be decomposed into squared bias, variance, and irreducible noise, so making a model more flexible lowers one term while raising another [8].

Figure 4 shows the trade-off on simulated data: 30 noisy training points drawn from a sine curve, fitted with polynomials of increasing degree (Section 6), and evaluated on 200 fresh validation points. A straight line (degree 1) underfits, with training RMSE 0.48 and validation RMSE 0.56. Degree 3 captures the shape, and validation RMSE drops to 0.29. Beyond that, training RMSE keeps falling, to 0.23 at degree 15, while validation RMSE *rises* to 0.38, and the degree-15 curve swings wildly near the edges of the data. Training error alone would always pick the most complex model. Validation error picks the one that generalizes.

In linear regression, complexity grows mainly with the *number of predictors*. With many predictors

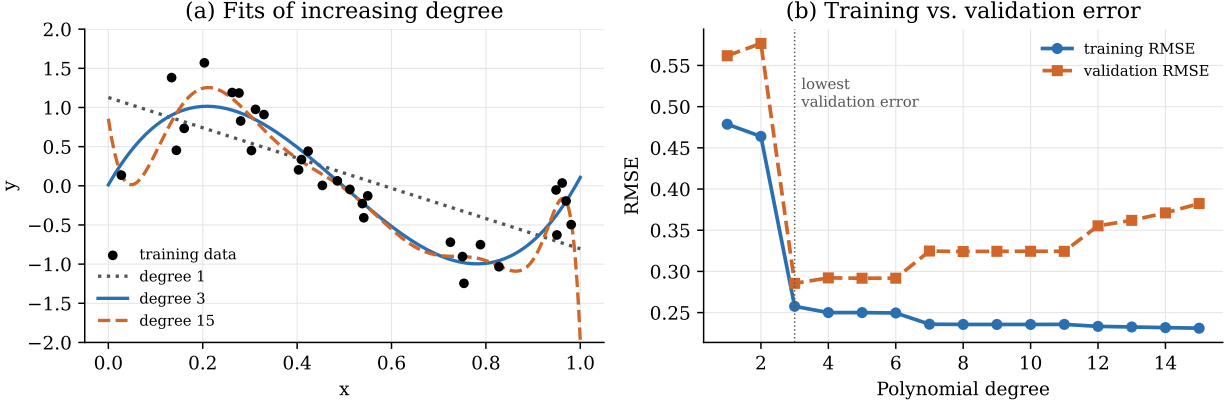


Figure 4: Underfitting and overfitting (simulated data). (a) Polynomial fits of degree 1, 3, and 15 to 30 training points. (b) Training RMSE keeps decreasing with degree; validation RMSE is lowest at degree 3 and then increases.

relative to observations — the $p \gg n$ setting of gene-expression data discussed in Part 2, Section 3.4 — OLS can fit the training data perfectly and generalize terribly. Multicollinearity is a related but distinct problem: even with few predictors, strongly correlated ones make individual coefficients high-variance. **Regularization** attacks both problems by adding a penalty on coefficient size to the least-squares objective.

5.2 Ridge, Lasso, and Elastic Net

Ridge regression [9] minimizes

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p b_j^2.$$

The L_2 penalty shrinks all coefficients toward zero; the intercept is not penalized. The tuning parameter $\lambda \geq 0$ sets the strength: $\lambda = 0$ gives OLS, and a very large λ pushes every coefficient toward zero. Ridge never sets a coefficient *exactly* to zero. Among correlated predictors it tends to share the weight, so their coefficients become smaller and much more stable, giving up a little bias for a large reduction in variance.

Lasso regression [20] replaces the squared penalty with absolute values:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |b_j|.$$

The L_1 penalty's sharp corner at zero lets the Lasso drive some coefficients *exactly* to zero, so it performs feature selection while it fits — the embedded method of Part 3, Section 1.4. Among a group of highly correlated predictors, however, it tends to keep one somewhat arbitrarily and drop the rest. Which one it keeps can change from sample to sample.

Elastic Net [23] combines the two penalties,

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left[\alpha \sum_j |b_j| + \frac{1-\alpha}{2} \sum_j b_j^2 \right],$$

where the mixing parameter $\alpha \in [0, 1]$ moves between Ridge ($\alpha = 0$) and Lasso ($\alpha = 1$). It keeps the Lasso’s ability to zero out coefficients while tending to keep or drop groups of correlated predictors together. In scikit-learn the penalty strength is called `alpha` and the mixing parameter `l1_ratio`; the library also scales the squared-error term by $1/(2n)$, so its `alpha` values are not numerically comparable to λ written as above [17].

	Ridge (L_2)	Lasso (L_1)	Elastic Net
Penalty	$\lambda \sum b_j^2$	$\lambda \sum b_j $	Weighted mix of both
Coefficients exactly 0?	No	Yes (sparse model)	Yes
Correlated predictors	Shares weight, stabilizes	Picks one, drops others	Tends to keep/drop as a group
Best when	Many small effects; collinearity	Few predictors truly matter	Correlated groups and sparsity wanted
Hyperparameters	λ	λ	λ, α

Table 2: Comparison of the three regularized linear models. All three require standardized predictors and a cross-validated choice of λ .

Key Idea

Regularization trades a small, deliberate bias for a reduction in variance. It helps most when predictors are many, noisy, or strongly correlated relative to the number of observations. When n is large and p is small, cross-validation will usually choose a tiny λ and the penalized model will behave like OLS — which is exactly what happens with the 16,512 training rows and 8 predictors in Section 9. The penalty strength λ is a *hyperparameter*: it is chosen by cross-validation on the training data, never by looking at the test set.

Why scaling is mandatory. The penalty treats every coefficient the same, but a coefficient’s size depends on its predictor’s units. Income measured in dollars needs a tiny coefficient; the same income in millions of dollars needs a coefficient a million times larger, which the penalty would punish far more. Without standardization the penalty therefore depends on arbitrary unit choices. Standardize every predictor first (`StandardScaler` inside the pipeline, fitted on training folds only) so that each coefficient is penalized on equal terms [12].

6 Polynomial and Nonlinear Relationships

Many relationships curve: yield rises with fertilizer and then levels off; energy demand is high on both cold and hot days. **Polynomial regression** captures curvature by adding powers of a predictor as new features:

$$\hat{y} = b_0 + b_1x + b_2x^2 + \cdots + b_kx^k.$$

The fitted curve is nonlinear in x , but the model is still *linear in its coefficients*: x^2 is just another column, and OLS or any regularized method fits it unchanged. In scikit-learn, `PolynomialFeatures` generates the powers and, for several predictors, all their pairwise products, i.e., interactions.

The degree k controls flexibility. Degree 2 or 3 is often enough. Higher degrees fit the training data ever more closely and, as Figure 4 showed, eventually overfit. They are also volatile at the edges of the data, where high powers grow explosively, which makes polynomial extrapolation especially

dangerous. The number of features also grows quickly: degree 2 on 8 predictors already gives 44 columns. Three practices keep polynomial models in check: choose the degree by cross-validation, pair polynomial features with Ridge regularization, and always plot the fitted curve against the data. Section 9 shows on real data how a degree-2 model can fail badly when an extreme predictor value gets squared. Truly nonlinear learners — regression trees, ensembles, kNN regression — are covered in later readings. The evaluation principles of Section 8 apply to them unchanged.

7 Developing a Regression Model

Figure 5 turns the KDD arc into a concrete modeling workflow. The dashed loop is important: diagnostics and validation results routinely send you back to data preparation.

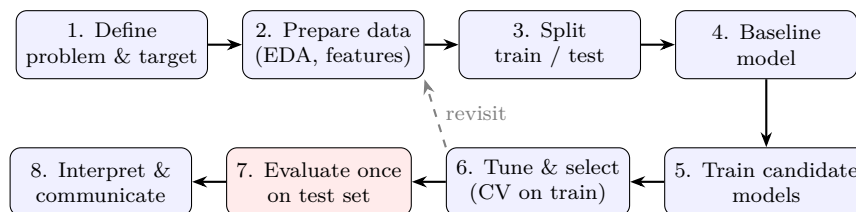


Figure 5: A regression modeling workflow. Everything up to step 6 uses training data only; the test set (red) is used exactly once.

Step 1 — Define the problem. State the target, its units, the unit of analysis (a patient, a field, a census block), and how the prediction will be used. “Predict next-day volumetric soil moisture at each sensor to schedule irrigation” implies very different data and error tolerances than “explain which soil properties relate to moisture retention.” Decide which predictors would actually be *available at prediction time*. A predictor recorded only after the outcome is a leak, however predictive it looks.

Step 2 — Prepare the data. Apply the preparation toolkit of the EDA readings: handle missing values (imputation is learned from training data), encode categorical variables as dummies, inspect outliers (Section 4.2), transform skewed variables (a log often helps both predictors and right-skewed targets such as prices or costs), and engineer features that encode domain knowledge (ratios, rates, seasonal indicators).

Step 3 — Split the data. Hold out a **test set** (commonly 20%) before doing any modeling. Use the remaining **training set** both to fit models and to compare them. Comparison needs **validation** data the model was not fitted on, either a separate validation split or, more efficiently, ***k*-fold cross-validation**: split the training set into k folds, fit on $k - 1$, score on the held-out fold, rotate, and average [14]. For time series or grouped data (repeated measurements per patient, multiple sensors per field), split by time or by group so that the validation data truly represent the future or unseen units.

Step 4 — Establish a baseline. Before any real model, score a trivial predictor, usually “always predict the training mean” (`DummyRegressor`). Its RMSE equals the target’s standard deviation and its R^2 is about zero. Every serious model must beat it clearly. A baseline turns “RMSE = 0.72” from an unanchored number into “38% lower error than knowing nothing.”

Step 5 — Train candidate models. Fit a small, purposeful set: OLS as the interpretable reference, a regularized variant, and perhaps a more flexible model (polynomial features, or a tree-based method from later readings). Each candidate should answer a question, e.g., “does curvature help?”, rather than add to a long list of algorithms.

Step 6 — Tune and validate. Hyperparameters — λ , the polynomial degree, the Elastic Net mixing — are chosen by cross-validation *inside* the training set (`RidgeCV`, `GridSearchCV`). Select the final model by its cross-validated error, and look at the spread across folds as well as the mean.

Step 7 — Evaluate on unseen data. Refit the selected model on the full training set and score it *once* on the test set. That number is your estimate of real-world performance. If you look at test scores while choosing among models, the test set has become a validation set and the estimate is optimistically biased [21].

Step 8 — Interpret and communicate. Report test MAE and RMSE in the target’s units, alongside the baseline. Explain the main coefficients (with the “held constant” qualifier and without causal language), show residual patterns, and state where the model should not be trusted: outside the training range, near the censoring cap, for subgroups with large residuals. Translate errors into consequences: “a typical miss of \$43k on a \$200k house.”

Common Pitfall

Leakage through preprocessing. Any step that *learns* from data — scaling (means and standard deviations), imputation (medians), encoding (category lists), feature selection (correlations with y), polynomial expansion followed by scaling — must be fitted on training data only and then applied to validation and test data. Standardizing the full dataset before splitting lets test-set statistics shape the model. This is the same leakage Part 1, Section 2.7 and Part 3, Section 1.5 warned against, and a recognized failure mode in practice [13]. The robust fix is mechanical: put every preprocessing step inside a scikit-learn `Pipeline`, so that cross-validation re-fits it within each training fold automatically.

8 Evaluation of Regression Models

Key Idea

Regression models should generally be evaluated on their ability to generalize to unseen data, not on how well they fit the training data. Every metric below should be computed on validation folds or the test set; training-set values are useful only for diagnosing over- or underfitting.

8.1 Error Metrics: MAE, MSE, and RMSE

The **mean absolute error**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

is the average size of a miss, in the units of y . It treats every unit of error equally, so it is relatively robust to a few large errors and easy to explain: “on average we are off by 43 points.”

The **mean squared error** and its square root, the **root mean squared error**,

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}},$$

square each error before averaging. MSE is in squared units ($\2), which is hard to interpret, but it is the quantity least squares minimizes and it has convenient mathematical properties. RMSE returns to the original units. Because of the squaring, both weight large errors heavily. Consider two models whose test errors are $(2, 2, 2, 2)$ and $(0, 0, 0, 8)$ (illustrative). Both have $\text{MAE} = 2$, but their RMSEs are 2 and 4. RMSE “notices” the single large miss; MAE does not. Always $\text{RMSE} \geq \text{MAE}$, and a large gap between them signals that a few big errors dominate.

Which to prefer? MAE describes typical error and resists outliers; RMSE suits settings where large errors are disproportionately costly (a badly underestimated flood level or drug dose) and is the natural criterion when errors are roughly normal. Their relative merits have been debated in the geoscientific modeling literature [22, 1]. The practical lesson is to report both and to choose the primary metric from the cost of errors in your application.

Percentage errors. The **mean absolute percentage error**, $\text{MAPE} = \frac{100\%}{n} \sum_i |(y_i - \hat{y}_i)/y_i|$, is popular because it is unit-free. It breaks down when actual values are zero or near zero: the division explodes or is undefined, and the metric penalizes over- and under-prediction asymmetrically [11]. Volumetric soil moisture of $0.05 \text{ m}^3/\text{m}^3$ in a dry field, predicted as 0.08, is a 60% error, even though the absolute miss is tiny. Use MAPE only for strictly positive targets that stay well away from zero, and even then report it alongside MAE.

8.2 Coefficient of Determination: R^2 and Adjusted R^2

The coefficient of determination compares the model’s squared error with that of the baseline that always predicts $\bar{y}$:

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}, \quad \text{SST} = \sum_{i=1}^n (y_i - \bar{y})^2.$$

On training data with an intercept, R^2 lies between 0 and 1 and is read as the *fraction of the target’s variance explained*. In the fertilizer example, $\text{SST} = 8.8$ and $\text{SSE} = 2.4$, so $R^2 = 1 - 2.4/8.8 = 0.73$. R^2 is unit-free, which makes it handy for comparing models of the same target. On *test* data, R^2 can be negative: the model predicts worse than the mean would.

Training R^2 has a well-known flaw: it can never decrease when a predictor is added, even a predictor of pure noise. **Adjusted R^2** corrects for this by penalizing the number of predictors p :

$$R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}.$$

It rises only if a new predictor improves the fit more than chance would. It is a useful quick check on training data, but it is not a substitute for validation.

Tip

Why a higher R^2 does not mean a better predictive model. On the 353 training patients of the diabetes case study (Section 10), OLS with the 10 real predictors has training $R^2 = 0.528$. Adding 30 columns of random noise raises training R^2 to 0.566. Adjusted R^2

falls slightly (0.514 to 0.510), and cross-validated R^2 falls sharply, from 0.491 to 0.422 (actual results). The “better” model by training R^2 is the worse predictor. R^2 also hides the size of errors in real units — an R^2 of 0.9 can still mean misses too large for the application — and it depends on how spread out y is in the sample, so values from different datasets are not comparable.

8.3 Choosing and Reporting Metrics

Metric	Measures	Units	Outliers	Use when / caveats
MAE	Typical absolute miss	Same as y	Robust	Communicating typical error; errors cost proportionally
MSE	Mean squared miss	y^2	Very sensitive	Optimization objective; hard to interpret directly
RMSE	Root of MSE	Same as y	Sensitive	Large errors are costly; compare with MAE for tail behavior
R^2	Variance explained vs. mean	Unit-free	Sensitive	Comparing models on the <i>same</i> data; never alone
Adj. R^2	R^2 penalized for p	Unit-free	Sensitive	Quick training-set check; not a validation metric
MAPE	Mean % miss	%	Unstable near 0	Positive targets far from zero only

Table 3: Regression evaluation metrics at a glance.

A good report gives at least one error metric in real units (MAE and/or RMSE), R^2 for context, the same numbers for the baseline, and some indication of variability (the standard deviation across cross-validation folds). It also includes residual plots, because two models with identical RMSE can fail in very different places.

9 Python Implementation

The example below runs the full workflow of Section 7 on the **California Housing** dataset [16], distributed with scikit-learn [17]. Each of its 20,640 rows is a census block group from the 1990 U.S. Census. The target `MedHouseVal` is the median house value in units of \$100,000. The eight predictors are median income (in \$10,000s), median house age, average rooms and bedrooms per household, population, average occupants per household, latitude, and longitude. The code uses pandas [15], NumPy [6], Matplotlib [10], and scikit-learn.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_validate, KFold
from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import (StandardScaler, PolynomialFeatures,
                                   FunctionTransformer)
from sklearn.dummy import DummyRegressor
```

```

from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

# 1-3. Load, inspect, and separate target from predictors
df = fetch_california_housing(as_frame=True).frame
print(df.shape) # (20640, 9)
print(df.describe().T[["mean", "min", "max"]].round(2))
X, y = df.drop(columns="MedHouseVal"), df["MedHouseVal"] # $100,000s

# 4. Minimal EDA: correlations with the target, and the capped target
print(df.corr()["MedHouseVal"].drop("MedHouseVal").round(2))
print("share of targets at the 5.0 cap:", round((y >= 5.0).mean(), 3))

# 5. One split; the test set is not touched again until step 9
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 6-7. Baseline and candidate models. All preprocessing lives inside a
# Pipeline, so it is re-fitted on the training folds only (no leakage)
skewed = ["AveRooms", "AveBedrms", "Population", "AveOccup"]
log_skewed = ColumnTransformer(
    [("log", FunctionTransformer(np.log1p), skewed)], remainder="passthrough")
alphas = np.logspace(-3, 3, 25)
models = {
    "Baseline (mean)": DummyRegressor(strategy="mean"),
    "OLS, raw": make_pipeline(StandardScaler(), LinearRegression()),
    "Poly(2)+Ridge, raw": make_pipeline(PolynomialFeatures(2, include_bias=False),
                                       StandardScaler(), RidgeCV(alphas=alphas)),
    "OLS, log": make_pipeline(log_skewed, StandardScaler(),
                              LinearRegression()),
    "Poly(2)+Ridge, log": make_pipeline(log_skewed,
                                       PolynomialFeatures(2, include_bias=False),
                                       StandardScaler(), RidgeCV(alphas=alphas)),
}

# Model selection: 5-fold cross-validation on the TRAINING set only
cv = KFold(n_splits=5, shuffle=True, random_state=0)
scoring = {"MAE": "neg_mean_absolute_error",
           "RMSE": "neg_root_mean_squared_error", "R2": "r2"}
rows = []
for name, model in models.items():
    s = cross_validate(model, X_train, y_train, cv=cv, scoring=scoring,
                      return_train_score=True)
    rows.append({"model": name,
                "train_RMSE": -s["train_RMSE"].mean(),
                "cv_MAE": -s["test_MAE"].mean(),
                "cv_RMSE": -s["test_RMSE"].mean(),
                "worst_fold_RMSE": -s["test_RMSE"].min(),
                "cv_R2": s["test_R2"].mean()})
print(pd.DataFrame(rows).round(3))

# 8-9. Refit on all training data; evaluate ONCE on the untouched test set
preds = {}
for name in ["Baseline (mean)", "OLS, raw", "Poly(2)+Ridge, log"]:

```

```

preds[name] = models[name].fit(X_train, y_train).predict(X_test)
print(f"{name:20s}"
      f" MAE={mean_absolute_error(y_test, preds[name]):.3f}"
      f" RMSE={root_mean_squared_error(y_test, preds[name]):.3f}"
      f" R2={r2_score(y_test, preds[name]):.3f}")

# 10. Residual plots for the simple linear model
fitted = preds["OLS, raw"]
resid = y_test - fitted
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].scatter(fitted, resid, s=4, alpha=0.3)
ax[0].axhline(0, color="k", lw=1)
ax[0].set(xlabel="Fitted value", ylabel="Residual")
ax[1].scatter(y_test, fitted, s=4, alpha=0.3)
ax[1].plot([0, 5], [0, 5], "k--", lw=1)
ax[1].set(xlabel="Actual", ylabel="Predicted")
plt.tight_layout()

# 12. Interpretation: standardized coefficients of the OLS model
ols = models["OLS, raw"] # already fitted in step 8
coef = pd.Series(ols[-1].coef_, index=X.columns)
print(coef.sort_values().round(3)) # $100k per 1-SD change, others fixed

```

What the code shows (actual output, scikit-learn 1.9). The EDA step finds that median income is by far the strongest single correlate of value ($r = 0.69$), that three ratio features have extreme maxima (up to 142 rooms and 1,243 occupants per household), and that 4.8% of targets sit at the cap. Table 4 gives the cross-validation results on the 16,512 training rows.

Model	Train RMSE	CV MAE	CV RMSE	Worst fold	CV R^2
Baseline (mean)	1.156	0.914	1.156	1.184	0.000
OLS, raw features	0.720	0.529	0.720	0.752	0.612
Poly(2) + Ridge, raw	0.690	0.524	1.499	4.501	-2.500
OLS, log ratios	0.673	0.487	0.673	0.706	0.660
Poly(2)+Ridge, log ratios	0.606	0.433	0.620	0.655	0.712

Table 4: Five-fold cross-validation on the California Housing training set (actual results; units of \$100,000 except R^2).

Four lessons stand out. First, every model clearly beats the baseline: OLS cuts RMSE by 38%. Second, the raw polynomial model has the *lowest training error* of the first three, yet its cross-validated RMSE is twice that of OLS, and its average R^2 is negative. Its MAE is fine (0.524), so it is not bad everywhere. In one fold it met the block group with 1,243 occupants per household, squared that extreme value, and predicted a median house value of about \$25.7 million. A single catastrophic prediction ruins RMSE and R^2 while barely moving MAE — a live demonstration of Section 8.1 and of the polynomial extrapolation warning in Section 6. Third, the remedy came from *data preparation*, not a fancier algorithm. Log-transforming the four skewed ratio features improves even plain OLS (CV RMSE 0.720 to 0.673) and makes the polynomial model both stable and the best overall. Fourth, RidgeCV on raw features selected a small penalty; with 16,512 rows and 8 predictors, regularization by itself changes little (a separate run confirmed Ridge and Lasso results identical to OLS to three decimals).

The selected model, Poly(2)+Ridge on log-transformed features, was then refitted on all training data and evaluated once on the 4,128 test block groups:

Model (test set)	MAE	RMSE	R^2
Baseline (mean)	0.906	1.145	0.000
OLS, raw	0.533	0.746	0.576
Poly(2) + Ridge, log (selected)	0.434	0.615	0.711

The selected model’s typical error is about \$43,000 per block group, against \$91,000 for the baseline. The test numbers agree closely with cross-validation for the selected model, which suggests the selection process did not overfit. The baseline and OLS rows are shown for reference only; they were not used to choose the model.

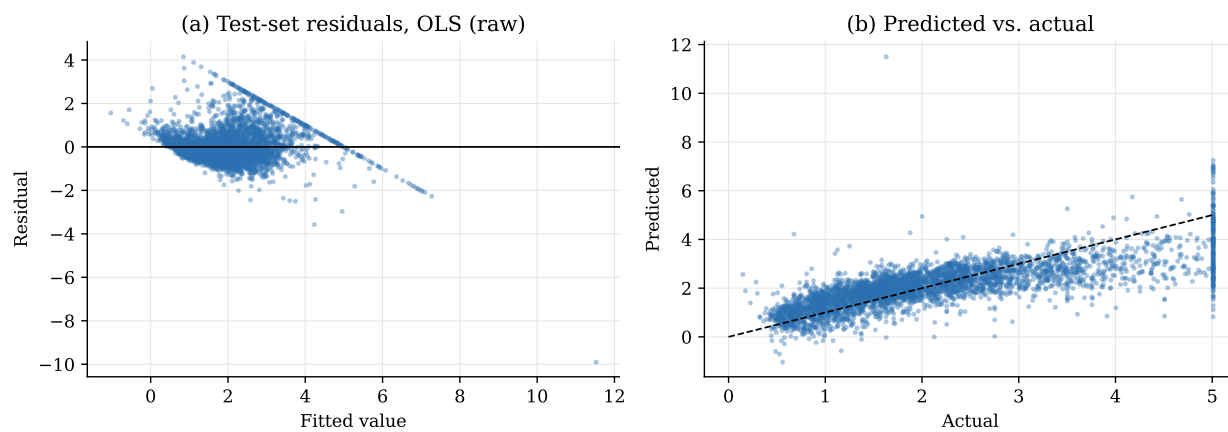


Figure 6: Output of step 10 (actual results): test-set residuals and predicted-versus-actual values for OLS on raw features. The vertical wall of points at Actual = 5 in (b) is the capped target, which the model systematically under-predicts; a handful of extreme block groups receive implausible predictions above 10.

Residuals and interpretation. Figure 6 repeats on the test set what Section 4.2 found in training. Residuals fan out as the fitted value grows, and the 184 capped test block groups are under-predicted by \$115,000 on average. The standardized OLS coefficients (in \$100k per one-standard-deviation change, others held fixed) are: median income +0.85, latitude -0.90 , longitude -0.87 , average bedrooms +0.34, average rooms -0.29 , house age +0.12, and population and occupancy close to zero. Income dominates, as the EDA suggested. Latitude and longitude jointly act as a stand-in for coastal location. The rooms/bedrooms signs illustrate the “held constant” rule from Section 3.2: at a fixed number of rooms, more of them being bedrooms is associated with higher value in this data, a partial association distorted by the two features’ collinearity ($VIF \approx 7-8$). None of these coefficients is a causal effect.

10 Integrated KDD Case Study: Predicting Diabetes Progression

Real-World Case Study

Problem. Clinicians would like to anticipate how fast a newly assessed diabetes patient’s disease will progress over the next year, so that follow-up intensity can be matched to risk, and to know which baseline measurements drive that prediction. This is a regression task with an explicitly dual goal: prediction *and* interpretation.

Data and target. The data are the 442 patients analyzed by Efron, Hastie, Johnstone, and Tibshirani [3] and distributed with scikit-learn as `load_diabetes`. The target is a quantitative measure of disease progression one year after baseline (range 25–346, mean 152, SD 77). There are ten baseline features: age, sex, body-mass index (BMI), average blood pressure (bp), and six blood-serum measurements `s1`–`s6`. According to the scikit-learn documentation these are total cholesterol (`s1`), LDL (`s2`), HDL (`s3`), total cholesterol/HDL ratio (`s4`), (likely) the log of serum triglycerides (`s5`), and blood sugar (`s6`). All results below are *actual results* from this real dataset with an 80/20 split (`random_state=42`; 353 training and 89 test patients) and 10-times-repeated 5-fold cross-validation on the training set; predictors were standardized inside each pipeline.

EDA. BMI ($r = 0.59$) and `s5` ($r = 0.57$) correlate most strongly with progression, followed by bp (0.44), `s4` (0.43), and `s3` (−0.39). The serum variables are strongly intercorrelated: `s1` and `s2` have $r = 0.90$, and their VIFs are 59.2 and 39.2 (`s3`: 15.4; `s5`: 10.1; `s4`: 8.9). In other words, total cholesterol is almost perfectly predictable from the other serum measurements — severe multicollinearity of the kind Section 4.2 warned about.

Models and evaluation. All four linear models clearly beat the mean baseline (CV RMSE 78.0) and are nearly indistinguishable from one another:

	CV MAE	CV RMSE (mean $\pm$ SD)	CV R^2	Test RMSE
Baseline (mean)	66.6	78.0 $\pm$ 4.7	−0.01	73.2
OLS	44.8	55.1 $\pm$ 4.1	0.49	53.9
Ridge	45.0	55.2 $\pm$ 3.9	0.49	53.8
Lasso	45.1	55.4 $\pm$ 3.9	0.49	52.9
Elastic Net	45.0	55.2 $\pm$ 3.9	0.49	53.5

The differences between models (a few tenths of a point) are an order of magnitude smaller than the fold-to-fold spread (about 4 points). Choosing a “winner” by these numbers alone would be over-interpretation, and the same holds for the slightly lower Lasso RMSE on only 89 test patients.

Interpretation — where the models differ. The coefficients tell a very different story from the error metrics (standardized units; progression points per 1 SD):

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
OLS	1.8	−11.5	25.6	16.8	−44.4	24.6	7.7	13.1	35.2	2.4
Ridge	1.8	−11.4	25.7	16.7	−33.6	16.2	2.9	11.6	31.0	2.5
Lasso	0	−8.2	26.2	15.2	−5.1	0	−11.1	0	22.1	1.8

OLS gives total cholesterol (`s1`) the largest coefficient of all, −44.4, offset by +24.6 for LDL (`s2`). A bootstrap of the training set shows how fragile this is: the `s1` coefficient has a standard deviation of 19.1 across resamples (`s2`: 15.5), against 4.2 for BMI. The two large, opposite-signed coefficients largely cancel and reflect collinearity, not biology. Ridge shrinks them; the Lasso (cross-validated penalty) removes age, `s2`, and `s4` entirely, reduces `s1` to −5.1,

and gives HDL (s3) the negative sign its correlation with progression suggests. Figure 7 traces each coefficient as the penalty grows. The coefficients that matter — BMI and s5 — stay large across penalties, while the collinear s1/s2 pair collapses toward zero.

Prediction. For a new patient one standard deviation above the training average in BMI and s5 and average on everything else, the Lasso model predicts $153.7 + 26.2 + 22.1 \approx 202$, about 48 points above the average patient. Given a typical error of about 45 points, this should be communicated as a risk indicator, not a precise forecast.

Knowledge and decision. The KDD outcome is not “the Lasso won.” It is three findings. (1) Routinely collected baseline measures explain roughly half the variance in one-year progression, a meaningful but far from complete picture. (2) Body-mass index, triglycerides (s5), and blood pressure are the most stable predictors across all model variants. (3) The individual roles of the cholesterol components cannot be separated in data this small and collinear, so claims about them should not be drawn from these coefficients. All three findings are associations in a single observational sample of 442 patients and would need external validation and a causal design before guiding treatment.

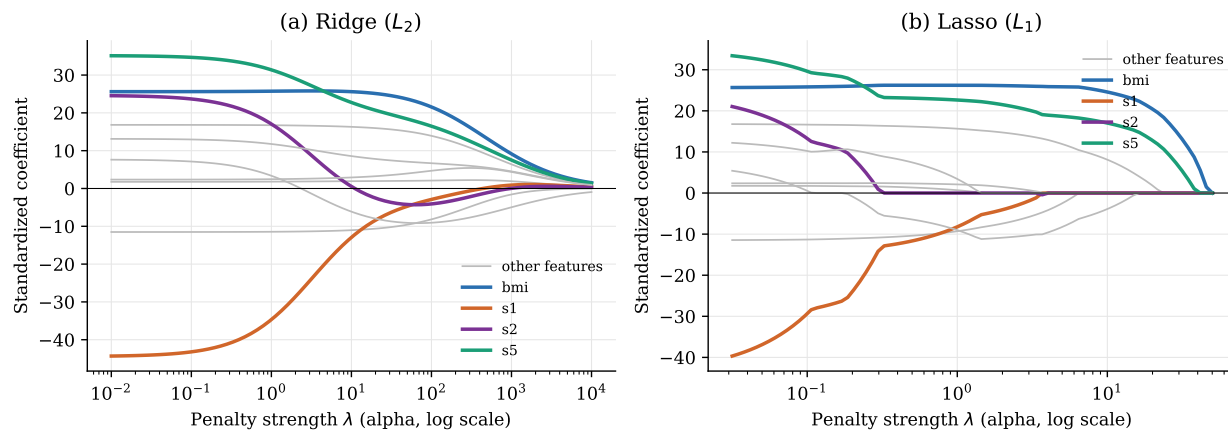


Figure 7: Coefficient paths for the diabetes training data (actual results). As the penalty grows (left to right), Ridge shrinks all coefficients smoothly, while the Lasso sets them to exactly zero one by one. The collinear s1/s2 pair (orange, purple) is large and opposite-signed under little penalty and collapses quickly; BMI and s5 persist.

11 Common Pitfalls

The mistakes below are the ones most often seen in student projects and in published work alike.

- **Evaluating only training performance.** Training error always rewards complexity (Figure 4; the raw polynomial model in Table 4). Always report validation or test error.
- **Data leakage.** Scaling, imputing, or selecting features on the full dataset, or using predictors recorded after the outcome. Keep preprocessing in a **Pipeline**; ask “would I have this value at prediction time?”
- **Using the test set during model selection.** Each look at test scores while choosing makes the final estimate more optimistic [21]. Choose with cross-validation; test once.

- **Overinterpreting R^2 .** Training R^2 rises with noise features; R^2 hides error size and depends on the spread of y . Pair it with MAE/RMSE in real units.
- **Ignoring residual patterns.** A good average error can hide systematic failure, such as the capped housing values or heteroscedastic costs. Plot residuals.
- **Ignoring multicollinearity.** Predictions survive it; interpretation does not. Check VIFs before reading coefficients, as in the $s1/s2$ example.
- **Extrapolating beyond the data.** Linear and especially polynomial models produce confident nonsense outside the training range.
- **Assuming correlation implies causation.** Coefficients are partial associations, conditional on what is in the model.
- **Inappropriate use of MAPE.** Targets at or near zero (dry soil, rare-event counts, small costs) make percentage errors explode.
- **Failing to scale features for regularized models.** Unscaled Ridge/Lasso penalties depend on arbitrary units.
- **Assuming a more complex model is better.** In the diabetes study four models tied; in the housing study the biggest gain came from a log transform. Prefer the simplest model whose validated performance is not meaningfully worse.

12 Key Takeaways

1. Regression is the predictive KDD task for continuous targets. It is a workflow — problem, preparation, EDA, features, modeling, evaluation, interpretation — not a single function call.
2. OLS chooses coefficients that minimize squared residuals; residuals are the starting point for both evaluation and diagnosis.
3. A multiple-regression coefficient is a partial association with all other predictors held constant. It is not a causal effect, and it can differ from the simple correlation.
4. Assumptions matter differently for prediction, interpretation, and inference. Residual, Q-Q, leverage, and VIF diagnostics reveal which are at risk.
5. Flexibility trades bias for variance. Training error always favors complexity; validation error reveals overfitting.
6. Ridge stabilizes correlated coefficients, the Lasso selects features, and Elastic Net blends both. All require standardized predictors and a cross-validated penalty.
7. Polynomial features add curvature while staying linear in the coefficients, but they amplify extreme values and are dangerous to extrapolate.
8. Fit every learned preprocessing step on training data only, select models with cross-validation, and use the test set exactly once.
9. Report MAE and RMSE in real units alongside a baseline. A large RMSE–MAE gap signals a few big errors; R^2 is context, not a verdict.

10. Good data preparation often beats algorithmic sophistication, and an honest statement of uncertainty and limits is part of the result.

Wrap-Up: Connecting Back

Almost every step in this reading reused an earlier one. Correlation analysis from Part 1 predicted that income would dominate the housing model and that the serum variables would be redundant. The leakage principle of Part 1, Section 2.7 became the rule that every preprocessing step lives inside a pipeline. VIF and the Lasso, introduced as feature-selection tools in Part 3, reappeared here as a diagnostic and as a regularized model. Standardization from the feature-scaling material turned out to be a requirement for any penalized model. The classification readings that follow will reuse the same workflow, splitting protocol, and baseline-first habit, changing only the metrics.

Review Questions

1. A hospital wants to predict (a) whether a patient will be readmitted within 30 days and (b) the number of days until readmission. Which is regression and which is classification? For (b), name one predictor that would be a leakage risk.
2. Using the five-plot fertilizer data of Section 2.3, compute SSE for the line $\hat{y} = 2 + 0.75x$ and confirm it exceeds 2.4. What does this illustrate about OLS?
3. In a model of annual healthcare cost, the coefficient on `age` is \$310 and on `num_chronic_conditions` is \$2,450. Write a precise one-sentence interpretation of the `age` coefficient, and explain why it should not be stated as “getting one year older causes costs to rise by \$310.”
4. A residual-versus-fitted plot shows residuals that are mostly negative at small and large fitted values and positive in the middle. Which assumption is violated, and name two ways you could address it.
5. Two predictors have VIFs of 45 and 38. Your manager wants only accurate predictions; your collaborator wants to report which of the two matters more. How does the high VIF affect each goal differently?
6. Explain why a Lasso model fitted to unstandardized features could select a different set of features if one predictor were changed from kilograms to grams.
7. Model A has test MAE = 4.1 and RMSE = 5.0; model B has MAE = 3.8 and RMSE = 9.6 on the same data. Describe what the error distributions probably look like, and say which you would deploy for dosing a medication.
8. A colleague reports training $R^2 = 0.97$ for a model with 120 predictors fitted on 150 samples. List three questions you would ask before believing the model is useful.
9. In Table 4, the raw polynomial model has the lowest training RMSE of the first three models but a negative CV R^2 , while its CV MAE is fine. Explain how all three facts can be true at once.
10. A soil-moisture model is evaluated with MAPE and reports 45% error, while its MAE is $0.02 \text{ m}^3/\text{m}^3$. Which number better describes performance, and why might MAPE be misleading here?

11. You tried six models, picked the one with the best test-set RMSE, and reported that RMSE. What is wrong with this procedure, and how would you fix it?
12. In the diabetes case study, all four models have nearly the same cross-validated error. Which would you recommend to a clinical team, and what would your recommendation depend on?

Suggested Hands-on Exercise

Regression modeling report (individual or pairs). Choose a regression dataset with a continuous target and at least eight candidate predictors, ideally from your project domain (health, bioinformatics, agriculture, environment, business), or use the instructor-provided dataset. Then:

1. **Define the prediction problem:** target, units, unit of analysis, intended use, and which predictors are available at prediction time.
2. **Perform focused EDA:** target distribution, correlations with the target, redundant predictor pairs, extreme values, and any capping or censoring.
3. **Prepare the data** inside a scikit-learn Pipeline (imputation, encoding, transformations, scaling), and split off a test set before modeling.
4. **Build at least two regression models** plus a mean baseline — for example OLS and one regularized or polynomial variant — tuning hyperparameters with cross-validation on the training set.
5. **Evaluate** with MAE, RMSE, and R^2 (cross-validated mean and SD), then evaluate the selected model once on the test set.
6. **Examine residuals:** residual-versus-fitted and Q–Q plots, the largest errors, and at least one subgroup comparison.
7. **Compare models** in a single table and state whether the differences exceed the fold-to-fold variability.
8. **Interpret** the selected model’s most important coefficients (with VIFs) without causal language.
9. **Recommend and justify** one model in a short paragraph addressed to a non-technical stakeholder, including where it should not be trusted.

Submit a notebook and a two-page summary. Grading emphasizes the reasoning — problem framing, leakage-free design, honest evaluation, and interpretation — over the number of models tried.

References

- [1] Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3), 1247–1250. <https://doi.org/10.5194/gmd-7-1247-2014>
- [2] Cook, R. D. (1977). Detection of influential observation in linear regression. *Technometrics*, 19(1), 15–18. <https://doi.org/10.1080/00401706.1977.10489493>
- [3] Efron, B., Hastie, T., Johnstone, I., & Tibshirani, R. (2004). Least angle regression. *The Annals of Statistics*, 32(2), 407–499. <https://doi.org/10.1214/009053604000000067>
- [4] Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54. <https://doi.org/10.1609/aimag.v17i3.1230>

- [5] Han, J., Pei, J., & Tong, H. (2023). *Data Mining: Concepts and Techniques* (4th ed.). Morgan Kaufmann.
- [6] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [7] Hasan, K. (2026). *Knowledge Discovery and Data Mining*. Grand Valley State University. Available at: https://github.com/mdkamrulhasan/data_mining_kdd
- [8] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- [9] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. <https://doi.org/10.1080/00401706.1970.10488634>
- [10] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [11] Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688. <https://doi.org/10.1016/j.ijforecast.2006.03.001>
- [12] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- [13] Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Article 15. <https://doi.org/10.1145/2382577.2382579>
- [14] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, Vol. 2, 1137–1145.
- [15] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- [16] Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3), 291–297. [https://doi.org/10.1016/S0167-7152\(96\)00140-X](https://doi.org/10.1016/S0167-7152(96)00140-X)
- [17] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [18] Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, 92–96. <https://doi.org/10.25080/Majora-92bf1922-011>
- [19] Shmueli, G. (2010). To explain or to predict? *Statistical Science*, 25(3), 289–310. <https://doi.org/10.1214/10-STS330>
- [20] Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>
- [21] Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91. <https://doi.org/10.1186/1471-2105-7-91>
- [22] Willmott, C. J., & Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Research*, 30(1), 79–82. <https://doi.org/10.3354/cr030079>
- [23] Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2), 301–320. <https://doi.org/10.1111/j.1467-9868.2005.00503.x>